

MCP Security: Structure, Tool Exposure, Auditing, and Deployment

What the Model Context Protocol (MCP) is

The Model Context Protocol is an open protocol that standardizes how applications provide context and capabilities to LLMs. It defines a client–server architecture where a **host** application (an IDE, a chat client, an agent runtime) runs one or more **MCP clients**, and each client maintains a 1:1 connection to an **MCP server**. The server exposes capabilities to the model: **tools** (functions the model can call), **resources** (read-only data the model can load into context), and **prompts** (reusable prompt templates).

Messages use JSON-RPC 2.0 over a transport. The two common transports are **stdio** (the host launches the server as a subprocess and talks over stdin/stdout) and **streamable HTTP / SSE** (the server is a network service). The transport choice is a primary security decision: stdio servers inherit the host user's local privileges; HTTP servers expose a network attack surface and need their own authN/Z.

How MCP servers are structured

A minimal server registers handlers and advertises them during the `initialize` handshake. Conceptually:

<code>initialize</code>	-> negotiate protocol version + capabilities
<code>tools/list</code>	-> server returns tool definitions (name, description, schema)
<code>tools/call</code>	-> host asks server to execute a tool with arguments
<code>resources/list, read</code>	-> server advertises and serves read-only context
<code>prompts/list, get</code>	-> server advertises reusable prompt templates

The server is just code. Each tool is a function with a JSON Schema describing its inputs. The danger is that **the arguments to `tools/call` are ultimately chosen by a model that an attacker may be able to influence** (via prompt injection in the conversation, in retrieved documents, or in other tool outputs). Treat every tool argument as untrusted input crossing a trust boundary.

How tool definitions are exposed to models

`tools/list` returns, per tool: a name, a natural-language description, and an `inputSchema` (JSON Schema). The host injects these definitions into the model's context so it can decide when and how to call them. Two consequences:

1. **The description is part of your attack surface.** A malicious or compromised server can put injection payloads in tool descriptions ("tool poisoning") — e.g. instructions telling the model to also exfiltrate data or call another tool. Because descriptions are model-visible context, they can hijack behavior.
2. **Schema is advisory, not enforcement.** The model is *asked* to conform to the schema; nothing stops a buggy or adversarial path from sending malformed arguments. The server must validate every

argument itself.

Auditing an MCP server codebase for injection vulnerabilities

The pattern to hunt for: **does a value derived from a tool argument reach a dangerous sink without validation?** Sinks to grep/AST-scan for:

Sink	Vulnerability class	CWE
<code>os.system, subprocess(..., shell=True)</code>	OS command injection	CWE-78
<code>eval, exec, compile</code>	Code injection	CWE-95
<code>pickle.loads, yaml.load</code>	Insecure deserialization	CWE-502
<code>open, raw path joins</code>	Path traversal	CWE-22
<code>requests.get(user_url)</code>	SSRF (cloud metadata!)	CWE-918
<code>f-string / % SQL into cursor.execute</code>	SQL injection	CWE-89
<code>Template(user_input)</code>	Server-side template inj.	CWE-1336

This repo ships an AST-based scanner that performs exactly this taint analysis:

```
python -m airte.mcp_audit.scanner path/to/server.py --fail-on HIGH
```

It identifies tool handlers (by `@tool`-style decorators or `*_tool/handle_*` naming), seeds taint from handler parameters, propagates through assignments and f-strings, and reports when tainted data reaches a sink. See `examples/vulnerable_mcp_server.py` for a labeled corpus of each bug class.

The scanner emits **SARIF 2.1.0** (`--format sarif`) for GitHub code scanning, and ships as a `pre-commit` hook (`airte-mcp-scan`) so the audit runs on every commit touching Python files.

Beyond static analysis, an audit also checks: Does the server authenticate the caller? Are tools authorized per-principal? Are tool descriptions free of embedded instructions? Is outbound network access allow-listed? Are secrets read from a vault rather than `env`/plaintext? Is every action audit-logged?

Secure MCP deployment patterns for enterprises

1. **Pin and vet servers.** Only run servers from sources you trust; pin versions by digest. A third-party MCP server is a supply-chain dependency (OWASP LLM03) with code execution in your environment.
2. **Least privilege per server.** Run each server with the minimum OS/cloud permissions it needs. For stdio servers, that means a dedicated low-privilege user or container; for HTTP servers, a scoped service identity.

3. **Authenticated, authorized tool calls.** Bind every session to a verified principal and enforce per-tool scopes. `airte.secure_mcp_server.auth` provides `Principal`, `ToolPolicy`, and `authorize()` primitives.
4. **Validate and confine inputs.** Schema-validate arguments, confine file paths to an allow-listed root (`validate_path`), and allow-list outbound URLs while blocking link-local/metadata ranges (`validate_url`).
5. **Isolate the runtime.** Sandbox servers (container, gVisor/Firecracker, seccomp) so a compromised tool cannot pivot. Deny egress by default.
6. **Human-in-the-loop for high-risk tools.** Destructive or irreversible actions require explicit approval (see `airte.guardrails.agent_guardrails`).
7. **Centralized audit logging.** Emit a structured event per tool call (subject, tool, arguments hash, outcome). The reference `SecureToolRegistry` does this.
8. **Treat tool descriptions as code.** Review them in PRs; reject embedded instructions; consider stripping/normalizing server-provided descriptions before they enter model context.

See `mcp-architecture-review.md` for the review checklist to apply before onboarding any MCP server.